ECE 2305 – Introduction to C Programming
Programming Tutorial 20
Iterative Calculations

The value of the *Golden Ratio* (φ) can be calculated using an iterative method. Let φ_{old} be your initial guess of the value. A value of $\varphi = 1$ is a good initial guess. A closer approximation can be calculated from

$$\varphi_{new} = \sqrt{\varphi_{old} + 1}$$

As this calculation is repeated, the new value gets closer to the golden ratio. This is called an iterative calculation. After each iteration, the *difference* (Δ_φ) between the new value and the old value can be calculated.

$$\Delta_\varphi = |\varphi_{new} - \varphi_{old}|$$

The process is completed when the *difference* is smaller than a desired value, perhaps $\Delta_\varphi < 0.000001$.

Use a **`while`** loop to iteratively calculate the *Golden Ratio* using a user input initial guess and user input difference. Count and display the number of iterations required.

An alternative iterative calculation for the *Golden Ratio* is

$$\varphi_{new} = 1 + \frac{1}{\varphi_{old}}$$

Compare this with the first calculation and determine which converges faster.

Input

```
// ECE 2305 - Tutorial 20
// Iterative Calculations
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    // Variables to store values
    double phi_old, phi_new, difference, tolerance;
    int count1 = 0, count2 = 0;

    // Getting user input
    cout << "Enter an initial guess for the Golden Ratio: ";
    cin >> phi_old;
    cout << "Enter the desired accuracy (e.g., 0.000001): ";
    cin >> tolerance;

    // First method: sqrt(phi_old + 1)
    do {
        phi_new = sqrt(phi_old + 1);
        difference = fabs(phi_new - phi_old);
        phi_old = phi_new;
        count1++;
```

```
    } while (difference >= tolerance);

    cout << "Method 1 (sqrt(phi_old + 1)) took " << count1 << " iterations to
converge.\n";

    // Reset values for the second method
    phi_old = 1; // Reset initial guess
    count2 = 0;

    // Second method: 1 + 1 / phi_old
    do {
        phi_new = 1 + 1 / phi_old;
        difference = fabs(phi_new - phi_old);
        phi_old = phi_new;
        count2++;
    } while (difference >= tolerance);

    cout << "Method 2 (1 + 1 / phi_old) took " << count2 << " iterations to
converge.\n";

    // Compare results
    if (count1 < count2) {
        cout << "Method 1 converged faster.\n";
    }
    else if (count2 < count1) {
        cout << "Method 2 converged faster.\n";
    }
    else {
        cout << "Both methods took the same number of iterations.\n";
    }

    system("pause");
    return 0;
}
```

Output

